

PES University
Department of Computer Science & Engineering(AIML)

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #42

Internal Microservice Catalog & Health Portal

Developer Tools & IT Operations

Deliverables:

1. Complete Requirements Table (5 FRs + 2 NFRs)
2. UML Use-Case Diagram
3. Use-Case Flow Specification

Deliverable 1: Complete Requirements Table

This section documents all functional and non-functional requirements for the Internal Microservice Catalog & Health Portal system.

1.1 Functional Requirements (FR-001 to FR-005)

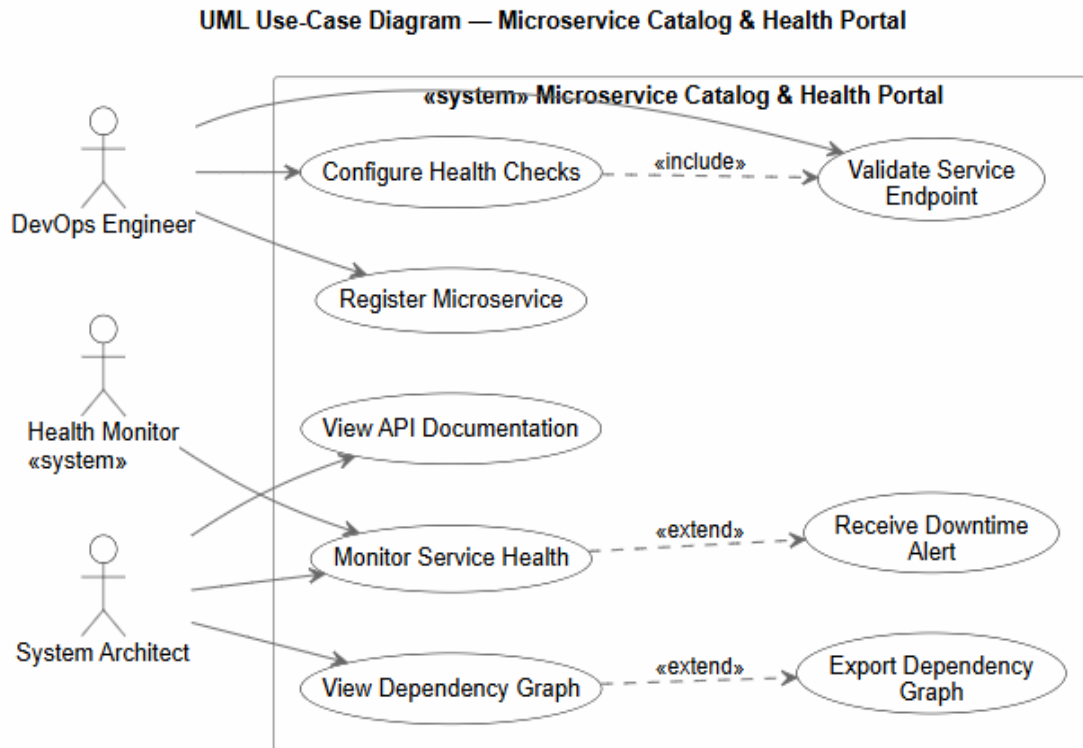
ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Monitoring	The system shall ping registered microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	High	Pass: Downtime is recorded and an alert is dispatched after 3 consecutive failed health probes.	Enables continuous monitoring of service health and availability.
FR-002	Visualization	The system shall render an interactive dependency graph viewer that maps upstream and downstream relationships between all catalogued microservices.	High	Pass: Graph correctly displays nodes and edges for all services.	Helps architects understand service dependencies and impact analysis.
FR-003	Documentation Management	The system shall aggregate and version API documentation from all registered microservices into a unified searchable portal.	Medium	Pass: Documentation is auto-fetched and searchable.	Provides a centralized source of API information for developers.
FR-004	Alerting	The system shall dispatch configurable alerts when service health thresholds are breached or downtime is detected.	High	Pass: Alerts delivered within threshold.	Ensures rapid response to failures and minimizes downtime.
FR-005	CRUD / Catalog Management	The system shall provide CRUD operations for registering, updating, and decommissioning microservices.	Medium	Pass: Registration and updates propagate correctly.	Maintains an accurate and up-to-date service inventory.

1.2 Non-Functional Requirements (NFR-001 & NFR-002)

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Scalability	The dependency graph viewer must render interactive node architectures containing up to 200 services smoothly with pan, zoom, and filter interactions.	High	Pass: Initial render < 2s and >= 30 fps during interaction.	Ensures usability and responsiveness as the system scales.
NFR-002	Security & Reliability	The portal shall enforce role-based access control (RBAC) and maintain 99.9% uptime, with health data encrypted at rest and in transit.	High	Pass: No unauthorized access and 99.9% availability.	Protects sensitive operational data and ensures service reliability.

Deliverable 2: UML Use-Case Diagram

The following diagram models all actors, primary use cases, and includes at least one <<include>> and one <<extend>> relationship as required.



PlantUML Source Code

The following PlantUML code can be used to regenerate this diagram in any compatible renderer:

```
@startuml
!theme plain
skinparam backgroundColor transparent

left to right direction

actor "DevOps Engineer" as DevOps
actor "System Architect" as Architect
actor "Health Monitor"
<<system>>" as Monitor

rectangle "<<system>> Microservice Catalog & Health Portal" {
    usecase "Register Microservice" as UC1

    DevOps --> UC1
    DevOps --> UC2
    Monitor --> UC3
    Monitor --> UC4
    Architect --> UC4
    Architect --> UC5
    UC1 -.-> UC6 : «include»
    UC4 -.-> UC7 : «extend»
    UC5 -.-> UC8 : «extend»
}
```

```
        usecase "Configure Health Checks" as UC2
        usecase "Validate Service
Endpoint" as UC3
        usecase "View Dependency Graph" as UC4
        usecase "View API Documentation" as UC5
        usecase "Monitor Service Health" as UC6
        usecase "Receive Downtime
Alert" as UC7
        usecase "Export Dependency
Graph" as UC8
    }
```

```
DevOps --> UC1
DevOps --> UC2
DevOps --> UC3
```

```
Architect --> UC4
Architect --> UC5
Architect --> UC6
```

```
Monitor --> UC6
```

```
UC2 ..> UC3 : <<include>>
UC6 ..> UC7 : <<extend>>
UC4 ..> UC8 : <<extend>>
```

```
@enduml
```

Deliverable 3: Use-Case Flow Specification

UC-006: Monitor Service Health

Attribute	Detail
Use-Case ID	UC-006
Use-Case Name	Monitor Service Health
Primary Actor	Health Monitor (<<system>>)
Secondary Actors	DevOps Engineer, Notification Service
Trigger	Scheduled timer event (every 30 seconds) or manual invocation by DevOps Engineer

Preconditions

1. The target microservice is registered in the catalog with a valid health endpoint URL.
2. The Health Monitor subsystem is active and network-reachable.
3. Alert channels (email/Slack/PagerDuty) are configured for the service owner.

Postconditions

Success: Health status is updated in the dashboard; rolling 24-hour availability percentage is recalculated.

Failure: Service is marked unreachable; downtime incident is logged; alert is dispatched after 3 consecutive failures.

Main Success Scenario

Step	Actor Action	System Response
1	Health Monitor initiates a scheduled health check cycle.	System retrieves the list of registered microservices and their health endpoint URLs from the Service Catalog.
2	Health Monitor sends an HTTP GET request to the microservice's /health endpoint.	System records the request timestamp and awaits response (timeout: 5 seconds).
3	Microservice responds with HTTP 200 OK and a JSON payload { "status": "UP" }.	System records the response timestamp, marks the service status as Healthy, and updates the last-seen timestamp.
4	Health Monitor computes the rolling 24-hour availability percentage.	System aggregates successful vs. total probes over the last 86,400 seconds and stores the updated metric.
5	DevOps Engineer views the Health Dashboard.	System renders the current health status, availability percentage, and historical uptime graph.

Alternate Flow A1: Service Unreachable (Single Failure)

Step	Actor Action	System Response
A1.1	At step 2, the microservice does not respond within 5 seconds or returns HTTP 5xx.	System marks the current probe as Failed, increments the consecutive failure counter to 1, and logs the incident with timestamp and error code.
A1.2	—	System does not dispatch an alert (threshold not yet reached).
A1.3	—	The next scheduled health check proceeds as per the Main Success Scenario.

Alternate Flow A2: Service Down — Alert Triggered (3 Consecutive Failures)

Step	Actor Action	System Response
A2.1	At step 2 of the third consecutive check, the microservice again fails to respond.	System marks the service status as Unreachable, sets downtime_start, and increments the consecutive failure counter to 3.
A2.2	—	System creates a Downtime Incident record with severity level derived from the service's configured SLA tier.
A2.3	—	System dispatches an alert via the configured channels (email, Slack, PagerDuty) to the service owner and on-call DevOps Engineer.
A2.4	—	The dashboard updates the service node color to red and reflects the new availability percentage.
A2.5	DevOps Engineer receives the alert and acknowledges it.	System logs the acknowledgment timestamp and stops redundant alert escalation for this incident.

Exception Flow E1: Endpoint Configuration Error

Step	Actor Action	System Response
E1.1	At step 2, the configured health endpoint URL is malformed or returns HTTP 404.	System logs a configuration error, marks the service as Misconfigured (not Unreachable), and notifies the DevOps Engineer via dashboard banner only (no paging alert).
E1.2	—	System skips availability computation for this service until the endpoint is corrected.

Business Rules

ID	Rule
BR-001	A service is considered down only after 3 consecutive failed probes; transient single failures are treated as warnings.
BR-002	Rolling availability is calculated as: $(\text{successful probes in last 24h} / \text{total probes in last 24h}) \times 100$.
BR-003	Alert suppression: If a DevOps Engineer acknowledges a downtime alert, no further alerts for the same incident are sent for 30 minutes unless the service recovers and fails again.